```
pip install matplotlib tensorflow keras mnist scikit-learn pandas opencv-python
opencv-contrib-python
```

Import packages and classes

```
# TensorFlow and tf.keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import optimizers
from tensorflow.keras import layers
from tensorflow.keras.models import load_model

# helper libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

print(tf.__version__)
```

Import the MNIST Dataset, waaarbij de 'fashion_mnist' de dataset is

```
mnist = keras.datasets.fashion_mnist
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

Let's explore the format (shape and size) of the dataset before training the model. dit doe je om te kijken of je test en train data gelijk zijn aan elkaar

```
# Print the shape of the training and test images & labels
print("Train images shape:", X_train.shape)
print("Train labels shape:", y_train.shape)
print("Test images shape:", X_test.shape)
print("Test labels shape:", y_test.shape)
```

Print de eerste en laatste 10 labels van de training en test set

```
# Print the first and last 10 labels from the training set
print("First 10 labels from the training set:", y_train[:10])
print("Last 10 labels from the training set:", y_train[-10:])

# Print the first and last 10 labels from the testing set
print("First 10 labels from the testing set:", y_test[:10])
print("Last 10 labels from the testing set:", y_test[-10:])
```

Het plotten van de 10 eerste van de test & trainingsset met de benaming er bij

```
# Plot 10 images from each set in a subplot, and print the true label (not the
numbers)
plt.figure(figsize=(10,10))

for i in range(20):
    plt.subplot(4,5,i+1) # 4 rows, 5 columns, cellnumber
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    if i < 10:
        plt.imshow(X_train[i], cmap='gray') # if you want them in inverted
colors: ax.imshow(X_train[i], cmap='binary'))
        label = plt.xlabel("Train:"+ class_names[y_train[i]]).set_color("green")
    else:
        plt.imshow(X_test[i], cmap='gray')
        label = plt.xlabel("Test:"+ class_names[y_test[i]]).set_color("red")
```

```
plt.show()

## Plotten van de nummers

plt.figure(figsize=(10,10))

for i in range(20):
    plt.subplot(4,5,i+1) # 4 rows, 5 columns, cellnumber
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    if i < 10:
        plt.imshow(X_train[i], cmap='gray') # if you want them in inverted
colors: ax.imshow(X_train[i], cmap='binary'))
        label = plt.xlabel("Train:"+ str(y_train[i])).set_color("green")
    else:
        plt.imshow(X_test[i], cmap='gray')
        label = plt.xlabel("Test:"+ str(y_test[i])).set_color("red")

plt.show()
```

Het preprocessen van de data en het builden van de modal met fotos preprcessen is dat de waarde gedeeld word door 225 om 1 of 0 te krijgen

```
# Define your sequential model, with the different layers, including the
preprocessing layers
model = keras.Sequential([
    # Preprocessing: Add a Rescaling layer to rescale the pixel values to the
[0, 1] range
    layers.Rescaling(1./255),
    # Input Layer: Add a Flattening layer to make 1-D vector of our 28x28 images
    layers.Flatten(),
    # Hidden layer: Add a Dense layer, aka a fully or densely connected layer of
128 neurons, and let them use the 'ReLu'-squishing or activation function
    layers.Dense(128, activation='relu'),
    # Output layer: Add a Dense layer of 10 neurons (because we have 10 possible
output labels), and link those neurons together in a group, via the 'softmax'-
activation function
    layers.Dense(10, activation='softmax')

])
```

Compilen van het model zelf, hierin zou je nog loss veranderen naar 'categorical_crossentropy'

```
# Compile the model with the optimizer, loss, and metrics
model.compile(optimizer=optimizers.Adam(learning_rate=0.001),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Het trainen van het model waarbij de epochs en batch sizes nog kunt aan passen. de validatie split is meestal 20%

```
history = model.fit(X_train, y_train, epochs=20, batch_size=50,
validation_split=0.2)
```

Het plotten van de Verliescurves & Nauwkeurigheidscurves

```
```python
    # Maak een figuur en een raster van subplots met een enkele aanroep
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10,5))
```
```

```
    # Plot de verliescurves op de eerste subplot
    ax1.plot(history.history['loss'], label='trainingsverlies')
    ax1.plot(history.history['val_loss'], label='validatieverlies')
    ax1.set_title('Verliescurves')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Verlies')
    ax1.legend()

    # Plot de nauwkeurigheidscurves op de tweede subplot
    ax2.plot(history.history['accuracy'], label='trainingsnauwkeurigheid')
    ax2.plot(history.history['val_accuracy'], label='validatienauwkeurigheid')
    ax2.set_title('Nauwkeurigheidscurves')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Nauwkeurigheid')
    ax2.legend()

    # Pas de ruimte tussen subplots aan
    fig.tight_layout()

    # Toon de figuur
    plt.show()
```


```
pip install pandas scikit-learn
pip install matplotlib
!pip install graphviz

!pip install pydotplus
```